

	EVELIA GIL PAREDES	DAW2B	CURSO 25/26	
	PROPUESTA PROYECTO FIN DE GRADO	1/10/25	VERSIÓN: 1.0	

BASE DE DATOS: Propuesta y Explicación

Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Web (DAW)

CURSO: 25/26

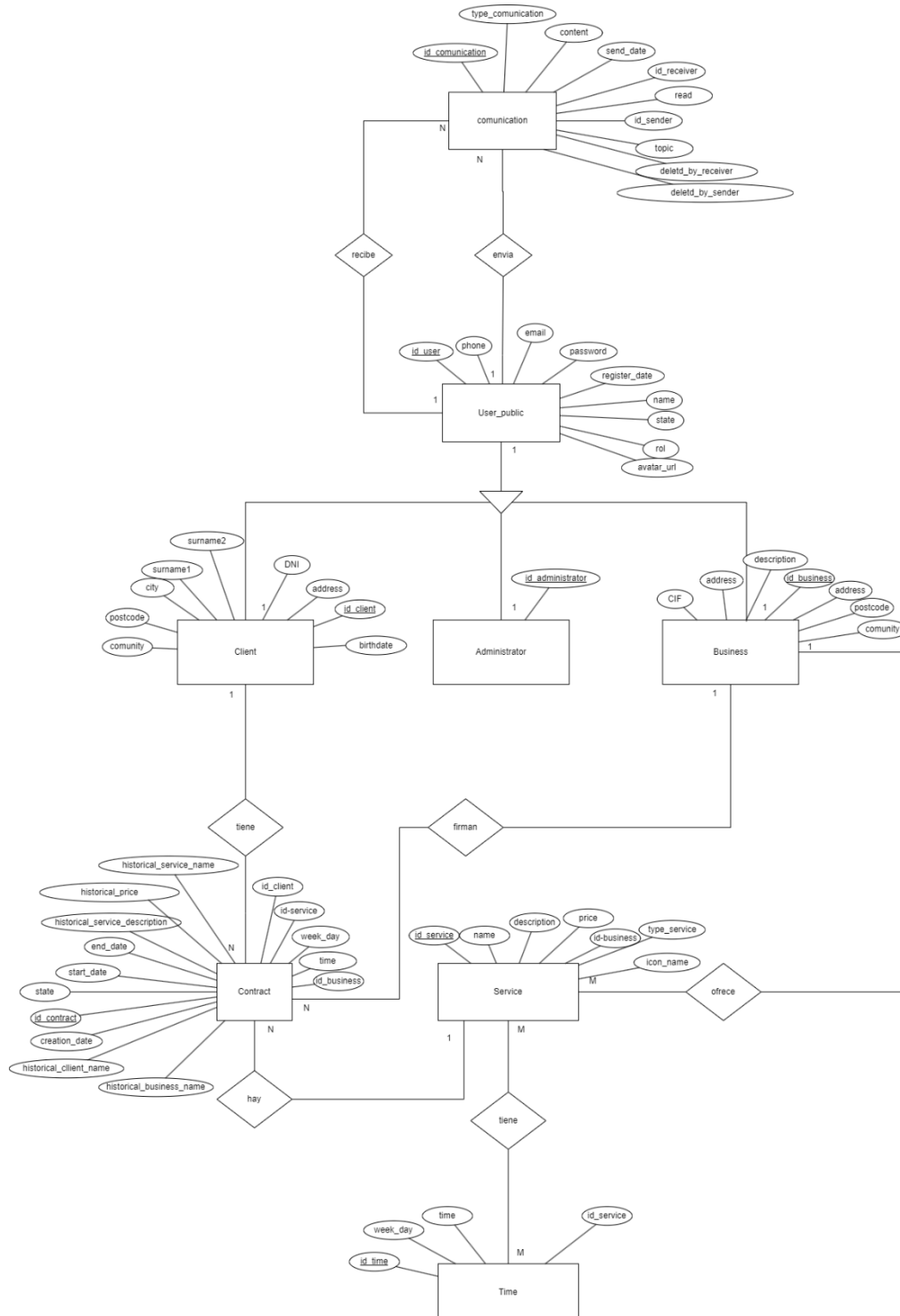
Alumno: Evelia Gil Paredes

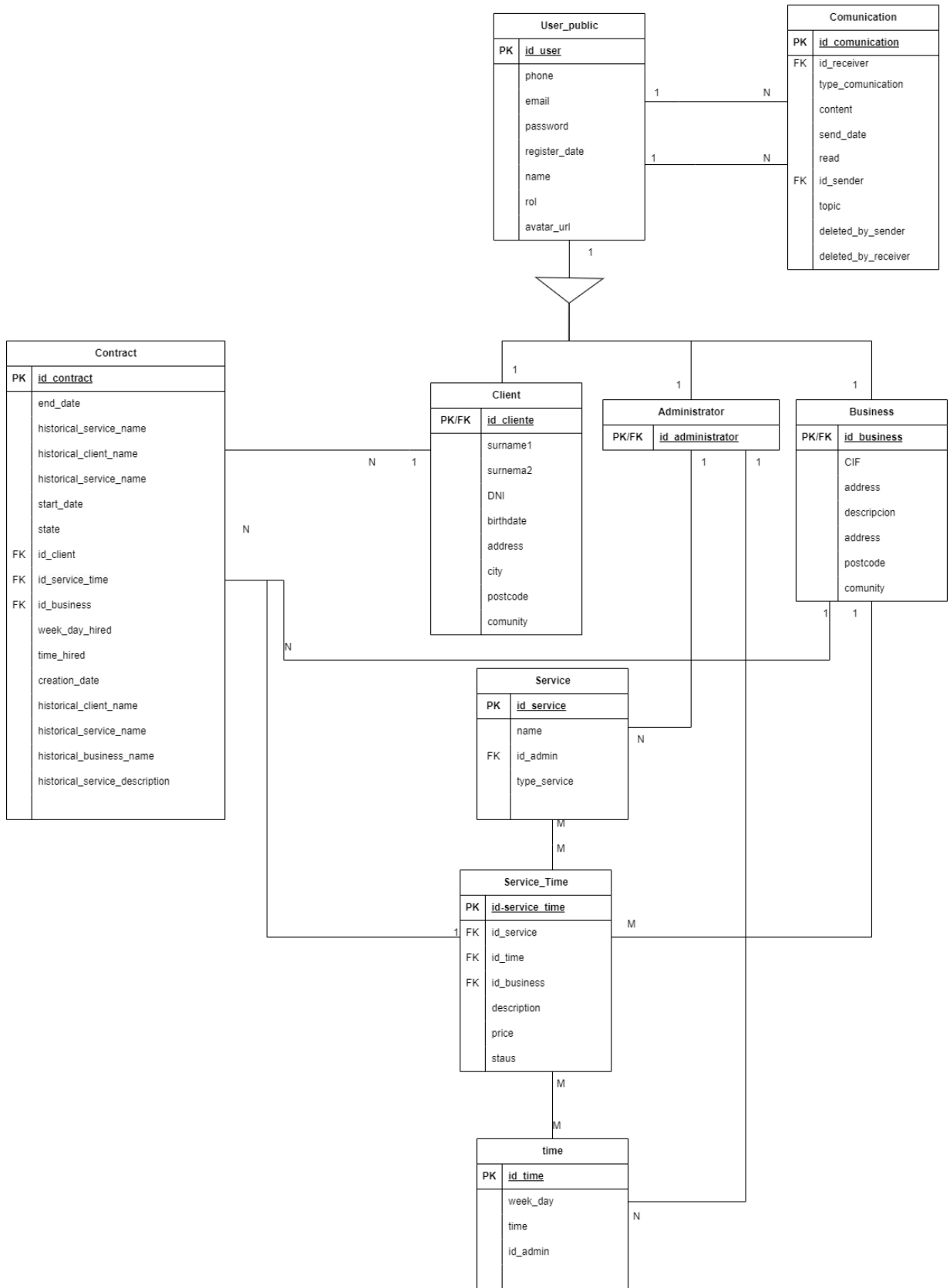
Fecha: Noviembre/2025

	EVELIA GIL PAREDES	DAW2B	CURSO 25/26	
	PROPUESTA PROYECTO FIN DE GRADO	1/10/25	VERSIÓN: 1.0	

El proyecto que se presenta tiene como objetivo la contratación de servicios de distintas categorías por parte de los cliente y la prestación de estos servicios por parte de la empresa.

Las tablas de la base de datos que se proponen son las siguientes:





	EVELIA GIL PAREDES	DAW2B	CURSO 25/26	
	PROPUESTA PROYECTO FIN DE GRADO	1/10/25	VERSIÓN: 1.0	

Justificación del diseño:

El diseño planteado persigue representar de manera adecuada el proceso de prestación de servicios bajo un horario determinado (día de la semana y hora) y su correspondiente contratación por parte de los clientes.

Los roles que interactúan con el sistema son:

- Cliente, que solicita o contrata servicios.
- Empresa, que ofrece dichos servicios.
- Administrador, que supervisa la información del sistema.

El diseño se ha enfocado en mantener la coherencia entre las entidades principales (empresa, cliente, servicio, horario y contrato), garantizando al mismo tiempo claridad en las relaciones y facilidad de consulta.

Supuestos:

- La base de datos propuesta es sencilla, pero refleja de manera adecuada la estructura lógica del aplicativo.
- Al ser un sistema nuevo, se espera un crecimiento progresivo, lo que implicará la creación futura de nuevas tablas (por ejemplo, *historiales*, *incidencias*, *pagos*, etc.).
- El diseño actual es escalable: permite incorporar más atributos a las entidades existentes o agregar nuevas entidades sin comprometer la integridad del modelo.

Principales decisiones:

La decisión principal fue la creación de la tabla Contrato, que actúa como nexo entre clientes, empresas y servicios ofrecidos en horarios específicos.

De este modo:

- Las empresas pueden conocer qué contratos tienen activos y en qué horarios prestan sus servicios.

	EVELIA GIL PAREDES	DAW2B	CURSO 25/26	
	PROPUESTA PROYECTO FIN DE GRADO	1/10/25	VERSIÓN: 1.0	

- Los clientes pueden visualizar los servicios contratados y cuándo serán prestados.

Esto resuelve la representación de la actividad simultánea tanto en el calendario del cliente como en el de la empresa.

Otra de las decisiones fue si la tabla intermedia entre servicio y horario iba a tener como clave primaria una clave compuesta por `id_service` e `id_time`. Al final se creó una clave primaria artificial autoincremental `id_service_time` y como FK `id_time` e `id_service`, como combinación única.

Ventajas de mi elección (PK artificial + UNIQUE):

1. Facilidad de referenciar
2. Mejor rendimiento en JOINS: los índices sobre un solo campo INTEGER son más eficientes y las búsquedas son más rápidas
3. Integridad mantenida
 - La restricción UNIQUE (`id_service`, `id_time`) sigue garantizando que no haya duplicados.

Finalmente, otra de las decisiones importantes fue el cambio de los atributos precio y descripción del servicio al `servicio_horario`, ya que se determinó que, un administrador podría crear un servicio y las empresa podría elegir entre un catálogo de servicios, pero era la empresa quien establecía la descripción y el precio para el servicio que esta ofrecía.

Dificultades y resolución:

1. Representación de las actividades

La necesidad de reflejar “cuándo y con quién” se realiza un servicio llevó a crear la tabla Contrato, que integra la información del cliente, la empresa y el horario contratado.

2. Cambios de horario y su impacto en los contratos

	EVELIA GIL PAREDES	DAW2B	CURSO 25/26	
	PROPUESTA PROYECTO FIN DE GRADO	1/10/25	VERSIÓN: 1.0	

Inicialmente, el horario del servicio (día y hora) dependía de las tablas Servicio_Horario y Horario.

Esto provocaba un problema: si el horario cambiaba, afectaba a todos los contratos relacionados, lo que no es deseable.

Para resolverlo, se añadieron los campos:

- day_week_hired
- time_hired

Estos campos representan una desnormalización controlada que viola la Tercera Forma Normal (3FN) debido a una dependencia transitiva (ya que la hora y el día dependen de id_service_time).

Sin embargo, esta desnormalización es intencionada y justificada por los siguientes motivos:

Motivo	Descripción
Simplicidad de consultas	Evita realizar múltiples JOIN con Service_Time y Time.
Estabilidad de datos históricos	Los contratos conservan la hora y día tal como se firmaron, aunque cambien los horarios de los servicios.
Rendimiento	Reduce el número de consultas anidadas, mejorando la velocidad de lectura.
Lógica de negocio	Los contratos son registros cerrados e inmutables, por lo que la redundancia no genera inconsistencias.

En una futura iteración del proyecto, se contempla la creación de una tabla Contract_copy ([Anexo I](#)) que almacene los valores originales de hora y día, cumpliendo con la 3FN.

Sin embargo, por simplicidad y alcance actual del sistema, la prioridad de dicha migración es baja.

	EVELIA GIL PAREDES	DAW2B	CURSO 25/26	
	PROPUESTA PROYECTO FIN DE GRADO	1/10/25	VERSIÓN: 1.0	

3. Crecimiento de la tabla Service_Time

Cada combinación de servicio y horario se representa como una tupla independiente en la tabla Service_Time.

Aunque esto puede generar un gran número de registros a futuro, la estructura es eficiente y clara:

- Facilita las consultas directas.
- No afecta el rendimiento ni la integridad del modelo.
- Mantiene la legibilidad y escalabilidad del sistema.

Política de modificación de contratos

Los contratos se consideran registros cerrados e inmutables.

No se permite su modificación una vez creados; si se desea cambiar alguna condición (horario, servicio, precio, etc.), se crea un nuevo contrato y se marca el anterior como *finalizado* o *cancelado*.

Esta política asegura:

- Integridad de la información contractual.
- Historial completo de acuerdos firmados.
- Simplicidad en la gestión de versiones y trazabilidad de los cambios.

	EVELIA GIL PAREDES	DAW2B	CURSO 25/26	
	PROPUESTA PROYECTO FIN DE GRADO	1/10/25	VERSIÓN: 1.0	

4. Campo rol en la tabla User_Public.

La decisión de incluir un campo explícito rol (o discriminador) en la tabla principal de Usuario, en lugar de depender exclusivamente de la existencia del usuario en tablas separadas (Cliente, Empresa, Administrador), representa una optimización técnica profunda.

A continuación, se detalla y justifica exhaustivamente esta elección arquitectónica desde una perspectiva de rendimiento, seguridad y mantenibilidad, redactada en tercera persona para su posible integración en documentación técnica o académica.

5. Campos históricos en la tabla Contract

En la tabla de contratos (contract) se han incluido algunos campos históricos («logs») para guardar datos necesarios que no deben ser borrados en caso de que ofertas, servicios y/o horarios se borren, salvo que se borre el contrato como tal.

Esto surge por la necesidad de mantener cierta información guardada por si fuera necesaria, además, se evitan así tener campos a null o sin información, que podría generar confusión.

1. Optimización Crítica de las Políticas de Seguridad (RLS en Supabase)

El sistema basa su infraestructura de backend en Supabase y PostgreSQL, haciendo uso intensivo de la Seguridad a Nivel de Fila (RLS, por sus siglas en inglés). Las políticas RLS se ejecutan en la base de datos por cada fila que un usuario intenta leer, insertar, modificar o eliminar.

Si el rol del usuario no está definido de forma explícita en la tabla Usuario, el motor de la base de datos se vería obligado a ejecutar subconsultas complejas (por ejemplo, comprobar con un EXISTS si el identificador del usuario se encuentra en la tabla Empresa, y luego en la tabla Cliente, etc.) para cada operación. Al centralizar el atributo rol en la tabla Usuario, la política RLS se simplifica a una evaluación condicional directa y atómica (ej. rol = 'Empresa'). Esto reduce drásticamente el coste computacional, evita cuellos de botella en el rendimiento y garantiza respuestas en

	EVELIA GIL PAREDES	DAW2B	CURSO 25/26	
	PROPUESTA PROYECTO FIN DE GRADO	1/10/25	VERSIÓN: 1.0	

milisegundos, algo vital cuando el volumen de datos (servicios, contratos, comunicaciones) comience a escalar.

2. Eficiencia en la Capa de Presentación (Angular y Gestión de Estado)

La arquitectura del cliente (Frontend) está desarrollada como una Single Page Application (SPA) en Angular, gestionando la navegación mediante *Guards* (rutas protegidas) e inyección de dependencias.

Cuando un individuo inicia sesión, el sistema necesita determinar de manera inmediata qué interfaz debe cargar (el Panel de Administración, el Panel de Empresa o el Perfil del Cliente). Si la base de datos careciera de un campo rol en la entidad principal, el AuthService de Angular tendría que realizar múltiples peticiones asíncronas para "adivinar" el tipo de usuario, o el backend tendría que ensamblar una respuesta artificial consultando varias tablas. Al disponer del campo rol directamente en el perfil base del usuario, el sistema obtiene esta información crucial en la primera y única petición de autenticación. Esto permite que los *Signals* de Angular actualicen el estado de la sesión instantáneamente y que el enrutador dirija al usuario a su panel correspondiente sin latencia perceptible.

3. Implementación del Patrón de Diseño de Base de Datos (Discriminador)

Desde el punto de vista del modelado de datos, las entidades Cliente, Empresa y Administrador comparten atributos y comportamientos comunes (credenciales, correo electrónico, teléfono, fecha de registro). Esto conforma un clásico esquema de herencia.

En bases de datos relacionales, al mapear una herencia mediante el patrón de "Tabla por Tipo" (donde existe una tabla padre y varias tablas hijas), es una práctica recomendada incluir una columna "discriminadora" en la tabla padre. El campo rol actúa exactamente como este discriminador. Su presencia indica inequívocamente a qué tabla hija (Cliente, Empresa o Administrador) pertenece el resto de la información específica de esa entidad. Esto elimina la ambigüedad y permite a los desarrolladores escribir consultas de combinación (JOIN) dinámicas y mucho más limpias, sin depender de lógicas deductivas frágiles.

	EVELIA GIL PAREDES	DAW2B	CURSO 25/26	
	PROPUESTA PROYECTO FIN DE GRADO	1/10/25	VERSIÓN: 1.0	

4. Escalabilidad y Flexibilidad Futura

El diseño de software debe anticipar la evolución del producto. Centralizar la asignación del rol en un atributo de la entidad Usuario dota al sistema de una flexibilidad superior.

Si en futuras iteraciones del proyecto CuidaDos se requiere añadir nuevos tipos de usuarios (por ejemplo, "Auditor", "Soporte Técnico" o "Gestor de Contenidos"), bastará con añadir un nuevo valor al campo rol (idealmente tipado como un ENUM en PostgreSQL). El sistema no requerirá la creación forzosa de nuevas tablas a menos que esos nuevos roles necesiten almacenar información estructuralmente distinta. Además, si en algún escenario un usuario necesitara cambiar de rol o tener un rol transitorio, la operación consistiría en una simple actualización de un campo de texto, en lugar de un complejo proceso de migración de registros entre distintas tablas.

ANEXOS

ANEXO I

Tabla Normalizada: Contrato

```
CREATE TABLE Contract (
    id_contract INT PRIMARY KEY AUTO_INCREMENT,
    id_client INT NOT NULL,
    id_business INT NOT NULL,
    id_service_time INT NOT NULL,
    start_date DATETIME NOT NULL,
    end_date DATETIME,
    hora_contratada TIME NOT NULL,
    day_week_hired VARCHAR(10) NOT NULL,
    state BOOLEAN DEFAULT TRUE, -- TRUE = activo, FALSE = finalizado
    FOREIGN KEY (id_client) REFERENCES Client(id_user),
    FOREIGN KEY (id_business) REFERENCES Business(id_user),
    FOREIGN KEY (id_service_time) REFERENCES Service_Time(id_service_time)
```

	EVELIA GIL PAREDES	DAW2B	CURSO 25/26	
	PROPUESTA PROYECTO FIN DE GRADO	1/10/25	VERSIÓN: 1.0	

);

Tabla Contrato_copia

```
CREATE TABLE Contract_Copy (
  id_contract INT PRIMARY KEY AUTO_INCREMENT,
  time_hired TIME NOT NULL,
  day_week_hired VARCHAR(10) NOT NULL,
  FOREIGN KEY (id_contract) REFERENCES Contract(id_contract)
);
```